

Manual de Usuario

Despacho 24 horas proyectado para el 15 de noviembre de 2027

1. Insumos

En el Excel “input_despacho24hv16” se encuentran los insumos que recibe el modelo para la construcción del escenario base y escenario 1.) “Baterías que no reducen limitación de inyección de renovables”. En el excel “input_despacho24hv16_sinrestriccionesfinas” se encuentran los insumos que recibe el modelo para la construcción del escenario 2.) “Baterías que reducen limitación de inyección de renovables”

En el excel “input_despacho24hv16_sinrestriccionesfinas” se encuentran los insumos que recibe el modelo para la simulación del escenario 2.) “Baterías que reducen limitación de inyección de renovables”.

En ambos documentos se encuentran las siguientes hojas:

Master_area:

Esta hoja describe el código de cada una de las 7 áreas y define si ésta es un área del sistema eléctrico colombiano o corresponde a un enlace de interconexión con otro país. Para el caso de esta simulación no se considera la coordinación con Ecuador.

Nombre Campo	Tipo de dato	Descripción
id_area	int	Para cada área del sistema se define un código - índice del Set A (áreas)
name_area	Str	Nombre del área (no entra a restricciones). Útil para reportes
type_area	Str	Internal: Área interna del sistema eléctrico External: Interconexión con otro país para el caso de análisis de importación y exportación o despacho coordinado.

Master_estacion_hidrologica:

Define el código de cada una de las 58 estaciones hidrológicas, asociándolas al embalse correspondiente. Esta asociación permite relacionar los caudales recibidos con el embalse asociado a cada planta.

Nombre Campo	Tipo de dato	Descripción
Id_estacionhidrologica	Int	Número de la estación hidrológica
Name_estacionhidrologica	Str	Nombre de la estación hidrológica (no entra a restricciones). Útil para reportes
Id_embalse	Str	Id embalse asociado a cada estación hidrológica. Índice del Set RES. Con esto se puede definir a que estación hidrológica

		pertenece cada embalse y, por lo tanto, que caudal recibe el embalse.
--	--	---

Caudalesm3s:

Describe los caudales asociados a cada estación hidrológica, previstos para el 15 de noviembre de 2027 mediante la aplicación de caudal histórico 2014-2016

Nombre Campo	Tipo de dato	Descripción
id_estacionhidrologica	Int	Número de estación hidrológica
Caudal_m3s	Float	Caudal promedio en m³/s. Parámetro Q[EH, w]. Decimal con coma.

Hydro_with_reservoir_base:

Describe los parámetros relacionados con 20 hidroeléctricas con embalse:

Nombre Campo	Tipo de dato	Descripción
Id_planta	Str	Índice del set HRES
Name_planta	Str	Nombre de la planta hidroeléctrica con embalse
Id_area	Int	Índice del área en la que se encuentra la planta de generación
Id_estacionhidrologica	Int	Número de la estación hidrológica
Id_embalse	Str	Id del embalse asociado a la hidroeléctrica
Pmax_MW	Float	Potencia máxima de la planta definida en MW
Prod_MW_per_m3s	Int	Factor de producción promedio [MW/m³/s]
Qmin_m3s	Int	Caudal turbinable mínimo [m³/s]
Qmax_m3s	Int	Caudal turbinable máximo [m³/s]
Vmin_hm3	Int	Volumen mínimo del embalse [Hm³]
Vmax_hm3	Int	Volumen máximo del embalse [Hm³]
Vinic_hm3	Int	Volumen inicial del embalse [Hm³]. Esto se calcula como volumen mínimo + [condición inicial en pu *(volumen máximo – volumen mínimo)]
O&M_USD\$_per_MWh	Int	Costo de Operación y Mantenimiento [USD/MWh]

Hydro_ror:

Describe los parámetros relacionados con 32 plantas hidroeléctricas sin embalse. 5 de estas plantas equivalen a 137 hidroeléctricas sin embalse que se encontraban en operación en 2025 en cada área:

Nombre Campo	Tipo de dato	Descripción
Id_planta	Str	Índice del set HROR
Name_planta	Str	Nombre de la planta sin embalse
Id_area	Int	Índice del área en la que se encuentra la planta de generación
Pmax_MW	Float	Potencia máxima de la planta definida en MW. Esta potencia ya limita al valor mínimo entre capacidad efectiva neta y caudal recibido* factor de producción promedio
Prod_MW_per_m3s	Int	Factor de producción promedio [MW/m ³ /s]
O&M_USD\$_per_MWh	Int	Costo de Operación y Mantenimiento [USD/MWh]

Profile_ror_by_area:

Se asume que el perfil horario respecto de la potencia máxima depende del área, por lo cual, se crea una tabla con el perfil en términos de fracción respecto de potencia máxima para cada una de las horas.

Nombre Campo	Tipo de dato	Descripción
Hour_in_day	int	Hora del día
Id_area	int	Índice del área en la que se encuentra la planta de generación
Profile	float	% de generación en la hora de la semana respecto de la potencia máxima. Se tomó como base el promedio de generación en 2025 de las hidroeléctricas sin embalse que componen el área correspondiente.

Termica_commitment_base:

Describe los parámetros relacionados con 40 termoeléctricas

Nombre Campo	Tipo de dato	Descripción
Id_planta	Str	Índice del set TC (térmicas con commitment)
Name_planta	Str	Nombre de la planta de generación termoeléctrica
Id_area	Int	Índice del área en el que se encuentra la termoeléctrica
Pmin_MW	Float	Potencia mínima de la planta [MW] – Para este ejercicio se eliminan los requerimientos mínimos por lo que Pmin=0.
Pmax_MW	Float	Potencia máxima de la planta definida en [MW]

Fuel code	Int	Código del combustible utilizado por la termoeléctrica
Heat_Rate	Float	Consumo específico [MBTU/MW]
O&M_USD\$_per_MWh	float	Costo variable operación y mantenimiento [USD/MWh]
Costo_transporte	Int	Costo de transporte [USD/MBTU] se coloca el valor en cero, considerando que el costo del combustible tomado para este análisis ya incluye el costo de transporte.
startup_cost_kUSD\$	Int	Costo de arranque [Kusd]. Al ser un despacho de 24 horas, no se utiliza este parámetro

Termica_no_commitment_base:

Describe los parámetros relacionados con 40 termoeléctricas

Nombre Campo	Tipo de dato	Descripción
Id_planta	Str	Índice del set TNC (térmicas no commitment)
Name_planta	Str	Nombre de la planta de generación termoeléctrica
Id_area	Int	Índice del área en el que se encuentra la termoeléctrica
Fuel code	Int	Código del combustible utilizado por la termoeléctrica
Pmin_MW	Float	Potencia mínima de la planta [MW] –Pmin=0.
Pmax_MW	Float	Potencia máxima de la planta definida en [MW]
O&M_USD\$_per_MWh	Float	Costo variable operación y mantenimiento [USD/MWh]
Heat_Rate	Float	Consumo específico [MBTU/MW]

Costo_combustible:

Nombre Campo	Tipo de dato	Descripción
Fuel code	Int	Código del combustible utilizado por la termoeléctrica
Fuel_name	Float	Nombre del combustible utilizado por la termoeléctrica.
Fuel_unit	Float	Unidad del combustible utilizado por la termoeléctrica [por defecto se utiliza MBTU]
Fuel_cost	Float	Costo de combustible [USD/MBTU]

Parametro_renovable:

Describe los parámetros relacionados con 349 plantas de generación renovable eólica y solar agrupadas en 128 plantas.

Nombre Campo	Tipo de dato	Descripción
Id_planta	Str	Índice del set REN (renovables)
Name_planta	Str	Nombre de la planta de generación eólica o solar
Id_area	Int	Índice del área en el que se encuentra la planta de generación
Pmax_MW	Float	Potencia máxima de la planta definida en [MW]
Tech	Str	Tipo de tecnología [Solar / Eólica]
Heat_Rate	Float	Consumo específico [MBTU/MW]
O&M_USD\$_per_MWh	float	Costo variable operación y mantenimiento [USD/MWh]

Profile_renewable:

En el caso de la energía eólica, se asume un perfil horario respecto de la potencia máxima para cada área el cual se calculó en función del promedio de generación de las plantas existentes en cada área. En relación con la tecnología eólica, se calculó un único perfil para la tecnología con base en la generación de 2025 de las plantas de generación

Nombre Campo	Tipo de dato	Descripción
Hour_in_day	int	Hora del día
Id_area	int	Índice del área en la que se encuentra la planta de generación
Tech	Str	Tipo de tecnología [Solar / Eólica]
Shape_Frac	Float	% de generación en la hora de la semana respecto de la potencia máxima [valor entre 0 y 1].

Limites_areas:

Describe los límites de intercambio entre áreas de forma que se cumpla la ecuación: $LB[MW] \leq \text{intercambio neto} \leq UB[MW]$. Si el intercambio es positivo, el área está importando y, si el intercambio es negativo el área, está exportando. Se toman los límites actuales con el fin de no considerar el escenario más conservador.

Nombre Campo	Tipo de dato	Descripción
Id_area	Int	Índice del área en la que se encuentra la planta de generación
LB_MW	Int	Límite inferior de intercambio que se corresponde con el límite de exportación del área [MW]
UB_MW	Int	Límite superior de intercambio que se corresponde con el límite de importación del área [MW]

Demanda:

Contiene el registro de las demandas horarias previstas para cada área.

Nombre Campo	Tipo de dato	Descripción
Hour_in_day	int	Hora del día
Id_area	int	Índice del área
Demanda	Float	Demanda para cada una de las horas [MW]

2. Descripción general del script

El script está diseñado para la simulación del despacho económico de un solo día con Pyomo, leyendo un LIBRO EXCEL (múltiples hojas).

2.1. Definiciones e importaciones:

Adicional a la preparación del entorno Python para poder hacer uso de la licencia académica de gurobi, el script inicia con la importación de módulos fundamentales de la biblioteca de Python. El módulo **re** permite buscar y limpiar texto, **typing** se usa para indicar el tipo de datos que se espera manejar, **pandas** para leer archivos de Excel y manejar tablas y **numpy** para operaciones numéricas, manejo de arrays, conversiones y valores nulos (NaN).

Luego se realiza las importaciones asociadas al modelo. **ConcreteModel** es donde se define el problema de optimización indicando los conjuntos - **Set**, variables - **Var**, restricciones – **Constraint**, función objetivo – **Objective** - que en nuestro caso corresponde a minimizar el costo del despacho. **Suffix** lo usamos para poder extraer la variable dual asociada al costo marginal.

```
from __future__ import annotations

import re
from typing import Dict, List, Optional, Tuple
import pandas as pd
import numpy as np

from pyomo.environ import (
    ConcreteModel, Set, Var, Constraint, Objective, Suffix,
    NonNegativeReals, Reals, minimize, SolverFactory, value
)
from pyomo.opt import TerminationCondition
```

2.2. Configuración:

Se carga el Excel que contiene los insumos del modelo y se nombran las hojas que contiene el Excel.

```
EXCEL_PATH = r"D:\beca fabio chaparro\tesis\modelizacion\STP Y SOC\analisis energetico25-30\input_despacho24hv16_sinrestriccionesfinas.xlsx"

SHEETS_REQUIRED = [
    "Master_area",
    "Master_estacion_hidrologica",
    "Caudalesm3s",
    "Parametros_globales",
    "Hydro_ror",
    "Profile_ror_by_area",
    "Hydro_with_reservoir_base",
    "Termica_commitment_base",
    "Termica_no_commitment_base",
    "Costo_combustible",
    "Parametro_renovable",
    "Profile_renovable",
    "Limites_areas",
    "Demanda",
    "Bess_base",
    "Req_regulation", # se carga pero NO se usa
    "Cap_renovable", # se carga pero NO se usa
]
```

2.3. Limpieza y conversión europea:

La función **def_clean_columns** permite limpiar los nombres de las columnas de cada una de las hojas eliminando todo lo que no sea letras, números y guiones bajos y pasando todo a minúscula. Se reemplaza los nombres originales de las columnas que trae el Excel por los nombres limpios. Ejemplo: PMax(MW) -> pmax_mw.

La función **def_to_numeric_eu** convierte a formato de punto como separador decimal. Ejemplo: 1.234,56 -> 1234.56. Por su parte, la función **coerce_numeric_columns** convierte columnas object (texto) a numérico con el fin de arreglar cualquier número que esté aun mal formateado, sin ajustar las columnas skip_cols.

La función **def_postprocess_types** revisa todas las hojas posibles que podría significar hora y los convierte a Int64.

```
def _clean_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Limpia nombres de columnas y los normaliza a snake_case lower."""
    df = df.copy()
    cols = []
    for c in df.columns:
        c = str(c).strip()
        c = re.sub(r"\s+", "_", c)
        c = re.sub(r"[^0-9a-zA-Z_]+", "", c)
        cols.append(c.lower())
    df.columns = cols
    return df
```

```

def _to_numeric_eu(series: pd.Series) -> pd.Series:
    """
    Convierte formato EU:
    - "1.234,56" -> 1234.56
    - "112,20" -> 112.2
    """
    if pd.api.types.is_numeric_dtype(series):
        return series
    s = series.astype(str).str.strip()
    s = s.replace({"": np.nan, "nan": np.nan, "None": np.nan})
    s = s.str.replace(".", "", regex=False).str.replace(",", ".", regex=False)
    return pd.to_numeric(s, errors="coerce")

def coerce_numeric_columns(df: pd.DataFrame, skip_cols: Optional[List[str]] = None) -> pd.DataFrame:
    """Convierte columnas object a numérico EU, excepto columnas en skip_cols."""
    df = df.copy()
    skip_cols = set([c.lower() for c in (skip_cols or [])])

    for c in df.columns:
        if c.lower() in skip_cols:
            continue
        if df[c].dtype == object:
            converted = _to_numeric_eu(df[c])
            if converted.notna().sum() > 0:
                df[c] = converted

    return df

skip_cols_by_sheet: Dict[str, List[str]] = {
    "Master_area": ["id_area", "name_area", "type_area"],
    "Master_estacion_hidrologica": ["id_estacionhidrologica", "name_estacionhidrologica", "id_embalse"],
    "Parametros_globales": ["name_parameter"],
    "caudalesm3s": ["id_estacionhidrologica"],
    "Hydro_ror": ["id_planta", "name_planta", "id_area"],
    "Profile_ror_by_area": ["id_area"],
    "Hydro_with_reservoir_base": ["id_planta", "name_planta", "id_area", "id_estacionhidrologica", "id_embalse"],
    "Termica_commitment_base": ["id_planta", "name_planta", "id_area", "fuel_code"],
    "Termica_no_commitment_base": ["id_planta", "name_planta", "id_area", "fuel_code"],
    "Costo_combustible": ["fuel_code", "fuel_name", "fuel_unit"],
    "parametro_renovable": ["id_planta", "name_planta", "id_area", "tech"],
    "Profile_renovable": ["id_area", "tech"],
    "Limites_areas": ["id_area", "scenario"],
    "demanda": ["id_area"],
    "bess_base": ["id_bess", "name_bess", "id_area"],
    "cap_renovable": ["id_planta", "id_area"],
}

```



```
def _postprocess_types(df: pd.DataFrame) -> pd.DataFrame: # Recibe un DataFrame y devuelve un DataFrame
    """Ajusta tipos básicos: IDs a str limpio y hora a Int64 si existe."""
    df = df.copy()

    str_cols = [
        "id_area", "name_area", "type_area", "id_estacionhidrologica", "name_estacionhidrologica", "id_embalse",
        "name_parameter", "id_planta", "name_planta",
        "id_hydro_ror", "id_hres",
        "id_tc", "id_tnc",
        "fuel_code",
        "id_ren", "id_planta",
        "id_bess",
        "tech", "plant_type",
        "name", "name_planta", "name_bess",
        "key"
    ]
    for col in str_cols:
        if col in df.columns:
            df[col] = df[col].astype(str).str.strip()

    for col in ["hour", "hour_in_day", "hora", "h"]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce").astype("Int64")

    return df
```

2.4. Carga Excel:

La función **def_normalize_sheet_name** convierte el nombre de la hoja a texto y lo deja sin espacios al inicio ni al final. La función **def_load_all_tables_from_excel** crea un diccionario donde la clave es el nombre normalizado de la hoja y el valor el nombre real de la misma.

```
def _normalize_sheet_name(s: str) -> str:
    return str(s).strip().lower()

def load_all_tables_from_excel(excel_path: str, sheets_required: List[str]) -> Dict[str, pd.DataFrame]:
    xls = pd.ExcelFile(excel_path, engine="openpyxl")
    sheet_map = {_normalize_sheet_name(sh): sh for sh in xls.sheet_names}

    data: Dict[str, pd.DataFrame] = {}

    for sh_req in sheets_required:
        key = _normalize_sheet_name(sh_req)

        aliases = [key]
        if key == "parametros_globales":
            aliases += ["parametros_globales"]
        if key == "parametros_globales":
            aliases += ["parametros_globales"]

        real_sheet = None
        for a in aliases:
            if a in sheet_map:
                real_sheet = sheet_map[a]
                break
        if real_sheet is None:
            raise ValueError(f"Falta la hoja requerida en el Excel: {sh_req}")
```

Con lo anterior, se lee el Excel y se aplica las funciones previamente mencionadas para guardar la hoja limpia usando el nombre normalizado como clave.

```

df = pd.read_excel(xls, sheet_name=real_sheet, engine="openpyxl")
    df = _clean_columns(df)

    skip = skip_cols_by_sheet.get(key, [])
    df = coerce_numeric_columns(df, skip_cols=skip)
    df = _postprocess_types(df)

    # fillna(0) para numéricas
    num_cols = [c for c in df.columns if pd.api.types.is_numeric_dtype(df[c])]
    if num_cols:
        df[num_cols] = df[num_cols].fillna(0)

    data[key] = df

    return data

```

2.5. Funciones auxiliares:

- **Def pick_first_existing** recibe el DataFrame y lista de nombres posibles de columna y devuelve el primer nombre que encuentra o None si no encontró ninguna.
- **Def get_global_panalties** obtiene las penalizaciones del DataFrame de parámetros globales y devuelve un diccionario. Si Excel no tiene los datos, define los valores por default para que el modelo no depende del Excel.
- **Def first_area_exchange_limits** devuelve un diccionario de los límites de intercambio por área (lb, lu).
- **Def build_inflow_by_reservoir** recibe el Data Frame de estación hidrológica que relaciona estación con el embalse y el Data Frame de caudales por estación hidrológica. Devuelve el caudal recibido por embalse.

3. Creación de diccionarios para usar en el modelo y solve:

Se cargan los DataFrame con nombres claros y cortos para cada uno y se crean los diccionarios y listas que se utilizan en el modelo de optimización.

4. Modelo PYOMO:

4.1. Conjuntos -Sets:

Se definen los conjuntos horas H, áreas A, flujo entre enlaces LINKS, hidroeléctricas sin embalse ROR, hidroeléctricas con embalse HRES, embalse RES térmicas commitment TC, térmicas no-commitment TNC, plantas de generación renovables eólica y solar REN y baterías BESS. En el caso de las horas se define que el conjunto tiene un orden de 1 a 24 para poder aplicar en el modelo la dinámica del embalse y de las baterías.

```

m.H = Set(initialize=H, ordered=True)
m.A = Set(initialize=AREAS)
m.LINKS = Set(initialize=LINKS, dimen=2)
m.ROR = Set(initialize=ROR)
m.HRES = Set(initialize=HRES)
m.RES = Set(initialize=RES)
m.TC = Set(initialize=TC)
m.TNC = Set(initialize=TNC)
m.REN = Set(initialize=REN)
m.BESS = Set(initialize=BEES)

```

- 4.2. **Variables:** Se definen las variables necesarias para el modelo usando el formato estándar `m.nombre_de_la_variable = Var(índice, dominio)`.

```

m.flow = Var(m.LINKS, m.H, domain=Reals)
m.exch = Var(m.A, m.H, domain=Reals)
m.load_shed = Var(m.A, m.H, domain=NonNegativeReals)
m.dump = Var(m.A, m.H, domain=NonNegativeReals)
m.p_ror = Var(m.ROR, m.H, domain=NonNegativeReals)
m.p_hres = Var(m.HRES, m.H, domain=NonNegativeReals)
m.p_tc = Var(m.TC, m.H, domain=NonNegativeReals)
m.p_tnc = Var(m.TNC, m.H, domain=NonNegativeReals)
m.p_ren = Var(m.REN, m.H, domain=NonNegativeReals)
m.curt = Var(m.REN, m.H, domain=NonNegativeReals)
m.q_turb = Var(m.HRES, m.H, domain=NonNegativeReals)
m.q_spill = Var(m.HRES, m.H, domain=NonNegativeReals)
m.v = Var(m.RES, m.H, domain=NonNegativeReals)
m.slack_vmin = Var(m.RES, m.H, domain=NonNegativeReals)
m.slack_vmax = Var(m.RES, m.H, domain=NonNegativeReals)
m.slack_vend = Var(m.RES, m.H, domain=NonNegativeReals)
m.p_dis = Var(m.BESS, m.H, domain=NonNegativeReals)
m.p_ch = Var(m.BESS, m.H, domain=NonNegativeReals)
m.soc = Var(m.BESS, m.H, domain=NonNegativeReals)
# m.y_ch = Var(m.BESS, m.H, domain=Binary)

```

- Flow: flujo de intercambio entre cada enlace LINKS en cada hora. En este caso no se definen flujos específicos porque no se tiene la información pero esta variable sirve para definir entre que áreas existe intercambio.
- Exch: flujo neto de intercambio del área a en la hora h
- Load_shed: Demanda no atendida del área A en la hora H
- Dump: Energía excedente que se bota en el área A en la hora H. Se utiliza para evaluar el funcionamiento del modelo
- P_ror: Potencia generada por la planta ROR en la hora H
- P_hres: Potencia generada por la planta HRES en la hora H
- P_tc: Potencia generada por la térmica TC en la hora H

- P_tnc: Potencia generada por la térmico TNC en la hora H
- P_ren: Potencia generada por la eólica/solar en la hora H
- Curt: Curtailment de energía renovable (eólica/solar) por restricciones
- Q_turb: caudal turbinado por central hidroeléctrica HRES en la hora H
- Q_spill: caudal vertido por central hidroeléctrica HRES en la hora H
- V: Volumen almacenado en el embalse asociado a HRES en la hora H
- Slack_vmin / slack_vmax: variables artificiales que relajan la restricción de volumen mínimo y máximo permitiendo violarlas en casos extremos. Esta variable se colocó siguiendo la recomendación de PSR (2009) para evitar infactibilidades
- Slack_vend: Variable que penaliza bajar el embalse al final del día por debajo del nivel del embalse inicial. Esta variable permite darle un valor al recurso hídrico
- P_dis: Potencia de descarga de la batería BESS en la hora H
- P_ch: Potencia de carga de la batería BESS en la hora H
- Soc: Estado de carga de la batería BESS en la hora H

4.3. **Restricciones:** Se definen las restricciones necesarias para el modelo

- Límite de intercambio por área: $\text{exch}[a,h] = \text{flujos que entran} - \text{flujos que salen}$.

Permite reconocer que las áreas reciben o exportan energía a áreas específicas, no necesariamente a todas las demás áreas, además de definir que si entra es positivo el flujo y si el flujo es de salida, entonces tiene un valor negativo

```
def exch_def_rule mdl, a, h):
    expr = 0.0
    for (i, j) in mdl.LINKS:
        if a == i:
            expr -= mdl.flow[i, j, h]
        elif a == j:
            expr += mdl.flow[i, j, h]
    return mdl.exch[a, h] == expr
m.ExchDef = Constraint(m.A, m.H, rule=exch_def_rule)
```

- Límite de intercambios de acuerdo con los límites de importación [UB] y exportación [LB] definidos

```

# Límite superior:  $exch[a, h] \leq ub$ 
def exch_ub_rule(mdl, a, h):
    lb, ub = exch_lim.get(a, (0.0, 0.0))
    return mdl.exch[a, h] <= ub
m.ExchUB = Constraint(m.A, m.H, rule=exch_ub_rule)

# Límite inferior:  $exch[a, h] \geq lb$ 
def exch_lb_rule(mdl, a, h):
    lb, ub = exch_lim.get(a, (0.0, 0.0))
    return mdl.exch[a, h] >= lb
m.ExchLB = Constraint(m.A, m.H, rule=exch_lb_rule)

```

- Conservación de la energía durante el intercambio: La suma de los intercambios entre las diferentes áreas debe ser igual a cero

```

def exch_zero(mdl, h):
    return sum(mdl.exch[a, h] for a in mdl.A) == 0
m.ExchZero = Constraint(m.H, rule=exch_zero)

```

- Ecuación de balance: la generación + Importación + descarga de las baterías debe ser igual a la demanda + exportación + carga de las baterías+ demanda no atendida

```

def balance_rule(mdl, a, h):
    gen_ror = sum(mdl.p_ror[r, h] for r in mdl.ROR if ror_area.get(r) == a)
    gen_hres = sum(mdl.p_hres[g, h] for g in mdl.HRES if hres_area.get(g) == a)
    gen_tc = sum(mdl.p_tc[g, h] for g in mdl.TC if tc_area.get(g) == a)
    gen_tnc = sum(mdl.p_tnc[g, h] for g in mdl.TNC if tnc_area.get(g) == a)
    gen_ren = sum(mdl.p_ren[g, h] for g in mdl.REN if ren_area.get(g) == a)

    bess_dis = sum(mdl.p_dis[b, h] for b in mdl.BESS if bess_area.get(b) == a)
    bess_ch = sum(mdl.p_ch[b, h] for b in mdl.BESS if bess_area.get(b) == a)

    return (
        gen_ror + gen_hres + gen_tc + gen_tnc + gen_ren
        + bess_dis - bess_ch
        + mdl.load_shed[a, h] - mdl.dump[a, h]      #
        - demand.get((a, h), 0.0)
        + mdl.exch[a, h]
        == 0
    )
m.Balance = Constraint(m.A, m.H, rule=balance_rule)

```

- Límites de potencias:

La potencia generada por cada planta ROR debe ser menor o igual a su potencia máxima afectada por el perfil de la hora.

```
def ror_limit mdl, r, h):
    a = ror_area.get(r, None)
    return mdl.p_ror[r, h] <= ror_pmax.get(r, 0.0) * ror_shape.get((a, h), 1.0)
m.RORLim = Constraint(m.ROR, m.H, rule=ror_limit)
```

La potencia generada por HRES debe ser igual a su caudal turbinado multiplicado por el factor medio de producción. Así mismo, la potencia generada debe estar entre su potencia mínima (se define cero por default) y su potencia máxima.

```
def hres_power_flow mdl, g, h):
    return mdl.p_hres[g, h] == hres_prod.get(g, 0.0) * mdl.q_turb[g, h]
m.HRESPowerFlow = Constraint(m.HRES, m.H, rule=hres_power_flow)
```

```
def hres_pmax_rule mdl, g, h):
    return mdl.p_hres[g, h] <= hres_pmax.get(g, 0.0)
m.HRESPmax = Constraint(m.HRES, m.H, rule=hres_pmax_rule)
```

La potencia generada por las térmicas no puede estar por debajo de su potencia mínima (cero por default ni ser superior a su potencia máxima).

```
def tc_pmax_rule mdl, g, h):
    return mdl.p_tc[g, h] <= tc_pmax.get(g, 0.0)

def tc_pmin_rule mdl, g, h):
    return mdl.p_tc[g, h] >= tc_pmin.get(g, 0.0)

m.TCPmax = Constraint(m.TC, m.H, rule=tc_pmax_rule)
m.TCPmin = Constraint(m.TC, m.H, rule=tc_pmin_rule)

def tnc_lim mdl, g, h):
    return (tnc_pmin.get(g, 0.0), mdl.p_tnc[g, h], tnc_pmax.get(g, 0.0))
m.TNCLim = Constraint(m.TNC, m.H, rule=tnc_lim)
```

La energía generada por la planta eólica y solar + el vertimiento renovable (curtailment) debe ser igual al recurso disponible para generar.

```
def ren_split mdl, g, h):
    avail = ren_pmax.get(g, 0.0) * ren_shape.get((g, h), 0.0)
    return mdl.p_ren[g, h] + mdl.curt[g, h] == avail
m.RenSplit = Constraint(m.REN, m.H, rule=ren_split)
```

- Caudales máximos y mínimos:

El caudal turbinado junto con el caudal vertido no puede superar el caudal máximo.

```
def hres_qmax_rule(mdl, g, h):
    return mdl.q_turb[g, h] + mdl.q_spill[g, h] <= hres_qmax.get(g, 0.0)

m.HRESQmax = Constraint(m.HRES, m.H, rule=hres_qmax_rule)
```

- Dinámica del embalse y límites:

El volumen del embalse al final de la hora 1 corresponde al volumen inicial más el caudal recibido menos el caudal turbinado y vertido. En el caso de las horas 2 a la 24 el volumen al final del embalse, toma el volumen de la hora anterior añadiendo el caudal recibido y restando el caudal turbinado y vertido.

```
def res_dyn(mdl, r, h):
    inflow = res_inflow_m3s.get(str(r), 0.0)
    outflow = sum(
        (mdl.q_turb[g, h] + mdl.q_spill[g, h])
        for g in mdl.HRES
        if hres_emb.get(str(g), None) == str(r)
    )

    gens = [g for g in mdl.HRES if hres_emb.get(str(g), None) == str(r)]
    vinic = hres_vinic.get(str(gens[0]), 0.0) if gens else 0.0

    if h == 1:
        return mdl.v[r, h] == vinic + (inflow - outflow) * FLOW_TO_HM3_PER_H
    return mdl.v[r, h] == mdl.v[r, h-1] + (inflow - outflow) * FLOW_TO_HM3_PER_H

m.ResDyn = Constraint(m.RES, m.H, rule=res_dyn)
```

Adicional a la dinámica se define las restricciones asociadas al límite del volumen de los embalses y la penalización para evitar que el nivel de cada embalse disminuya por debajo de su nivel inicial.

```

def res_vmax(mdl, r, h):
    gens = [g for g in mdl.HRES if hres_emb.get(str(g), "") == str(r)]
    vmax = hres_vmax.get(str(gens[0]), 0.0) if gens else 0.0
    return mdl.v[r, h] <= vmax + mdl.slack_vmax[r, h]

def res_vmin(mdl, r, h):
    gens = [g for g in mdl.HRES if hres_emb.get(str(g), "") == str(r)]
    vmin = hres_vmin.get(str(gens[0]), 0.0) if gens else 0.0
    return mdl.v[r, h] + mdl.slack_vmin[r, h] >= vmin

m.ResVmax = Constraint(m.RES, m.H, rule=res_vmax)
m.ResVmin = Constraint(m.RES, m.H, rule=res_vmin)

# Slack si volumen final queda por debajo del inicial ---
def vend_def(mdl, r):
    gens = [g for g in mdl.HRES if hres_emb.get(str(g), "") == str(r)]
    vinic = hres_vinic.get(str(gens[0]), 0.0) if gens else 0.0
    last_h = max(list(mdl.H))
    return mdl.slack_vend[r] >= vinic - mdl.v[r, last_h]

m.VendDef = Constraint(m.RES, rule=vend_def)

```

- Dinámica del almacenamiento de las baterías y límites de carga y descarga

La potencia cargada o descargada en cada hora no puede ser superior a su límite máximo

```

def bess_ch_energy_limit(mdl, b, h):
    emax = bess_emax.get(b, 0.0)
    eff_ch = bess_effch.get(b, 1.0)
    soc_max = bess_socmax.get(b, 1.0) * emax

    soc_prev = (bess_socini.get(b, 0.5) * emax) if h == 1 else mdl.soc[b, h-1]
    return mdl.p_ch[b, h] <= (soc_max - soc_prev) / max(eff_ch, 1e-6)

m.BESSChEnergy = Constraint(m.BESS, m.H, rule=bess_ch_energy_limit)

def bess_dis_energy_limit(mdl, b, h):
    emax = bess_emax.get(b, 0.0)
    eff_dis = max(bess_effdis.get(b, 1.0), 1e-6)
    soc_min = bess_socmin.get(b, 0.0) * emax

    soc_prev = (bess_socini.get(b, 0.5) * emax) if h == 1 else mdl.soc[b, h-1]

    return mdl.p_dis[b, h] <= (soc_prev - soc_min) * eff_dis

m.BESSDisEnergy = Constraint(m.BESS, m.H, rule=bess_dis_energy_limit)

```



```

def bess_ch_ub mdl, b, h):
    return mdl.p_ch[b, h] <= bess_pch.get(b, 0.0)

def bess_dis_ub mdl, b, h):
    return mdl.p_dis[b, h] <= bess_pdis.get(b, 0.0)

m.BESSChUB = Constraint(m.BESS, m.H, rule=bess_ch_ub)
m.BESSDisUB = Constraint(m.BESS, m.H, rule=bess_dis_ub)

```

El almacenamiento de la batería al final de la hora 1 corresponde al almacenamiento inicial sumando la carga y restando la descarga de la hora. Para las horas 2 a la 24, en lugar de tomar el almacenamiento inicial se toma como punto de partida el nivel de almacenamiento de la hora anterior.

```

def bess_soc_dyn mdl, b, h):
    emax = bess_emax.get(b, 0.0)
    eff_ch = bess_effch.get(b, 1.0)
    eff_dis = max(bess_effdis.get(b, 1.0), 1e-6)
    soc0 = bess_socini.get(b, 0.5) * emax
    if h == 1:
        return mdl.soc[b, h] == soc0 + eff_ch * mdl.p_ch[b, h] - (1.0 / eff_dis) * mdl.p_dis[b, h]
    return mdl.soc[b, h] == mdl.soc[b, h-1] + eff_ch * mdl.p_ch[b, h] - (1.0 / eff_dis) * mdl.p_dis[b, h]
m.BESSSOCDyn = Constraint(m.BESS, m.H, rule=bess_soc_dyn)

```

El estado de carga de la batería debe estar entre su mínimo y máximo permitido.

```

def bess_soc_bounds mdl, b, h):
    emax = bess_emax.get(b, 0.0)
    return (bess_socmin.get(b, 0.0) * emax, mdl.soc[b, h], bess_socmax.get(b, 1.0) * emax)
m.BESSSOCBounds = Constraint(m.BESS, m.H, rule=bess_soc_bounds)

```

Finalmente se define la restricción que indica que no se puede cargar la batería más de lo que le permite el estado de carga inicial en la hora y no se puede descargar la batería más de lo que se tiene disponible que se encuentra por encima del nivel mínimo.

```

def bess_ch_energy_limit mdl, b, h):
    emax = bess_emax.get(b, 0.0)
    eff_ch = bess_effch.get(b, 1.0)
    soc_max = bess_socmax.get(b, 1.0) * emax

    soc_prev = (bess_socini.get(b, 0.5) * emax) if h == 1 else mdl.soc[b, h-1]
    return mdl.p_ch[b, h] <= (soc_max - soc_prev) / max(eff_ch, 1e-6)

m.BESSChEnergy = Constraint(m.BESS, m.H, rule=bess_ch_energy_limit)

def bess_dis_energy_limit(mdl, b, h):
    emax = bess_emax.get(b, 0.0)
    eff_dis = max(bess_effdis.get(b, 1.0), 1e-6)
    soc_min = bess_socmin.get(b, 0.0) * emax

    soc_prev = (bess_socini.get(b, 0.5) * emax) if h == 1 else mdl.soc[b, h-1]

    return mdl.p_dis[b, h] <= (soc_prev - soc_min) * eff_dis

m.BESSDisEnergy = Constraint(m.BESS, m.H, rule=bess_dis_energy_limit)

```

4.4. Función objetivo y solución

Se calcula el costo horario de la térmica en función de su heat rate y el costo del combustible que utiliza.

```

def therm_cost(p_mw, hr, fuel_id, om):
    fc = fuel_cost.get(fuel_id, 0.0)
    return (hr * fc + om) * p_mw

```

Se define el costo que penaliza los vertimientos de las plantas hidroeléctricas

```
spill_cost_usd_per_hm3 = 100.0
```

Se define la función objetivo a minimizar:

```

def obj_rule(mdl):
    total = 0.0

    for g in mdl.TC:
        fid = tc_fid.get(g, "")
        for h in mdl.H:
            total += therm_cost(mdl.p_tc[g, h], tc_hr.get(g, 0.0), fid, tc_om.get(g, 0.0))
    for g in mdl.TNC:
        fid = tnc_fid.get(g, "")
        for h in mdl.H:
            total += therm_cost(mdl.p_tnc[g, h], tnc_hr.get(g, 0.0), fid, tnc_om.get(g, 0.0))
    for r in mdl.ROR:
        for h in mdl.H:
            total += ror_om.get(r, 0.0) * mdl.p_ror[r, h]
    for g in mdl.HRES:
        for h in mdl.H:
            total += hres_om.get(g, 0.01) * mdl.p_hres[g, h]
            total += spill_cost_usd_per_hm3 * (mdl.q_spill[g, h] * FLOW_TO_HM3_PER_H)
    for g in mdl.REN:
        for h in mdl.H:
            total += ren_om.get(g, 0.0) * mdl.p_ren[g, h]
            total += penalties["pen_curt"] * mdl.curt[g, h]
    c_deg = 0 # USD/MWh
    for b in mdl.BESS:
        for h in mdl.H:
            total += c_deg * (mdl.p_ch[b, h] + mdl.p_dis[b, h])
    for a in mdl.A:
        for h in mdl.H:
            total += penalties["pen_load_shed"] * mdl.load_shed[a, h]
            total += penalties["pen_dump"] * mdl.dump[a, h]
    for r in mdl.RES:
        for h in mdl.H:
            total += penalties["pen_slack_vmin"] * mdl.slack_vmin[r, h]
            total += penalties["pen_slack_vmax"] * mdl.slack_vmax[r, h]
    pen_vend = 5000.0
    for r in mdl.RES:
        total += pen_vend * mdl.slack_vend[r]

    return total

m.Obj = Objective(rule=obj_rule, sense=minimize)

```

Esta función objetivo corresponde a minimizar:

$$\begin{aligned}
& \sum (P_{Ter,h} \times Costo_{comb_{Ter}} \times HR_{Ter} + O\&M_{Ter}) \\
& + \sum (P_{ren,h} \times O\&M_{ren} + Curt_{ren,h} \times Costo_{curt}) \\
& + \sum (P_{hres,h} \times O\&M_{hres} + Spill_{hres,h} \times Costo_{spill}) \\
& + \sum (Slack_{vmax_v} \times Costo_{slackvmax}) \\
& + \sum (Slack_{vmin_v} \times Costo_{slackvmin}) \\
& + \sum (Min(0, vfin_{24} - vini_v) \times Costo_{vini < vfin}) \\
& + \sum (P_{hror,h} \times O\&M_{ror}) + \sum (DNA_a \times Costo_{DNA}
\end{aligned}$$

Donde:

$P_{Ter,h}$ Potencia generada por la planta térmica ter en la hora h [MW]

$Costo_{comb_{Ter}}$ Costo de combustible para la planta Ter [USD/MBTU]

HR_{Ter} Heat Rate de la térmica Ter [MBTU/MWh]

$O\&M_{Ter}$ Costo variable de operación y mantenimiento para la planta térmica Ter [USD/MWh]

$P_{ren,h}$ Potencia generada por la planta eólica o solar ter en la hora h [MW]

$O\&M_{ren}$ Costo O&M asignado a la planta renovable ren. Se asume un costo de 0,01 USD/MWh para todas las plantas.

$Curt_{ren,h}$ Vertimiento de energía eólica o solar

Finalmente, habiendo descrito la función objetivo a minimiza se soluciona utilizando la licencia académica de Gurobi y especificando las variables duales que se quieren obtener.

```

solver = SolverFactory("gurobi")
if not solver.available(False):
    raise RuntimeError("No encuentro GUROBI disponible.")

solver.options["TimeLimit"] = 3600
solver.options["Threads"] = 0

results = solver.solve(m, tee=False, suffixes=["dual"])

lmp = {(a, h): float(m.dual.get(m.Balance[a, h], np.nan)) for a in m.A for h in m.H}
dual_exch_ub = {(a,h): float(m.dual.get(m.ExchUB[a,h], np.nan)) for a in m.A for h in m.H}
dual_exch_lb = {(a,h): float(m.dual.get(m.ExchLB[a,h], np.nan)) for a in m.A for h in m.H}

```